# Week 2 Assignment

NumPy & Pandas for Data Analytics

| | |
|---|---|
| **Name** | Nikhil Kankatri |
| **College** | Alva's Institute of Engineering & Technology, Moodbidri |
| **Branch** | B.E. Computer Science & Engineering (IoT, Cybersecurity and Blockchain) |
| **Duration** | 1 Week |
| **Total Marks** | 100 |

# Contents

Task 1: NumPy Introduction & Arrays
Task 2: NumPy Indexing, Slicing & Reshaping
Task 3: NumPy Mathematical & Statistical Operations
Task 4: Pandas Series & DataFrame
Task 5: Reading & Inspecting Data
Task 6: Selecting, Filtering & Sorting Data
Task 7: Handling Missing Values
Task 8: Merge, Concatenate, GroupBy & Pivot Table
Task 9: Exporting Data
Task 10: Mini Project: Sales Data Analysis
Datasets Used

# Task 1: NumPy Introduction & Arrays

### Explanation

In this task I imported NumPy and made an array of 12 numbers. I printed the array along with its shape, size and data type. After that I made a simple one-dimensional array and a 3x3 two-dimensional array to see how the shape changes.

### Source code (task1_numpy_arrays.py)

```
# Task 1: NumPy Introduction & Arrays
import numpy as np

# Array with at least 10 numbers
arr = np.array([12, 45, 7, 23, 89, 34, 56, 11, 78, 90, 5, 67])
print("Array:", arr)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data type:", arr.dtype)

# One-dimensional array
arr_1d = np.array([1, 2, 3, 4, 5])
print("\n1-D array:", arr_1d)
print("1-D shape:", arr_1d.shape, "| dimensions:", arr_1d.ndim)

# Two-dimensional array
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("\n2-D array:\n", arr_2d)
print("2-D shape:", arr_2d.shape, "| size:", arr_2d.size,
      "| dtype:", arr_2d.dtype, "| dimensions:", arr_2d.ndim)
```

### Output

```
Array: [12 45  7 23 89 34 56 11 78 90  5 67]
Shape: (12,)
Size: 12
Data type: int64

1-D array: [1 2 3 4 5]
1-D shape: (5,) | dimensions: 1

2-D array:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
2-D shape: (3, 3) | size: 9 | dtype: int64 | dimensions: 2
```

# Task 2: NumPy Indexing, Slicing & Reshaping

### Explanation

I created an array of 12 numbers and used indexing to get the first, fifth and last elements. With slicing I took a range, every second element and the reversed array. Then I made a 2-D array and picked out one value, a row, a column and a small block. At the end I reshaped the 12-element array into 3x4, 4x3 and 2x2x3 and printed the original and reshaped versions.

### Source code (task2_numpy_indexing_slicing_reshaping.py)

```
# Task 2: NumPy Indexing, Slicing & Reshaping
import numpy as np

arr = np.arange(10, 130, 10)      # 12 elements: 10, 20, ..., 120
print("Original array:", arr)

# Indexing
print("\nFirst element arr[0]:", arr[0])
print("Fifth element arr[4]:", arr[4])
print("Last element arr[-1]:", arr[-1])

# Slicing
print("\nSlice arr[2:6]:", arr[2:6])
print("Every 2nd element arr[::2]:", arr[::2])
print("Reversed arr[::-1]:", arr[::-1])

# Two-dimensional array
arr_2d = np.array([[11, 12, 13, 14],
                   [21, 22, 23, 24],
                   [31, 32, 33, 34]])
print("\n2-D array:\n", arr_2d)
print("Element at row 1, col 2:", arr_2d[1, 2])
print("Row 0:", arr_2d[0])
print("Column 3:", arr_2d[:, 3])
print("Sub-matrix (rows 0-1, cols 1-2):\n", arr_2d[0:2, 1:3])

# Reshaping
print("\nOriginal 1-D (shape {}):".format(arr.shape), arr)
r1 = arr.reshape(3, 4)
print("\nReshaped to (3, 4):\n", r1)
r2 = arr.reshape(4, 3)
print("\nReshaped to (4, 3):\n", r2)
r3 = arr.reshape(2, 2, 3)
print("\nReshaped to (2, 2, 3):\n", r3)
print("\nFlattened back:", r1.flatten())
```

### Output

```
Original array: [ 10  20  30  40  50  60  70  80  90 100 110 120]

First element arr[0]: 10
Fifth element arr[4]: 50
Last element arr[-1]: 120

Slice arr[2:6]: [30 40 50 60]
Every 2nd element arr[::2]: [ 10  30  50  70  90 110]
Reversed arr[::-1]: [120 110 100  90  80  70  60  50  40  30  20  10]

2-D array:
 [[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
Element at row 1, col 2: 23
Row 0: [11 12 13 14]
Column 3: [14 24 34]
Sub-matrix (rows 0-1, cols 1-2):
 [[12 13]
 [22 23]]

Original 1-D (shape (12,)): [ 10  20  30  40  50  60  70  80  90 100 110 120]

Reshaped to (3, 4):
 [[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]

Reshaped to (4, 3):
 [[ 10  20  30]
 [ 40  50  60]
 [ 70  80  90]
 [100 110 120]]
```

```
Reshaped to (2, 2, 3):
 [[[ 10  20  30]
  [ 40  50  60]]

 [[ 70  80  90]
  [100 110 120]]]

Flattened back: [ 10  20  30  40  50  60  70  80  90 100 110 120]
```

# Task 3: NumPy Mathematical & Statistical Operations

### Explanation

I used the marks of 10 students in Maths and Science. First I added, subtracted, multiplied and divided the two arrays element by element. Then I used NumPy functions to find the mean, median, minimum, maximum, standard deviation and sum for each subject.

### Source code (task3_numpy_math_stats.py)

```
# Task 3: NumPy Mathematical & Statistical Operations
import numpy as np

# Dataset: marks of 10 students in two subjects
maths = np.array([78, 85, 62, 90, 55, 73, 88, 67, 94, 81])
science = np.array([72, 80, 70, 85, 60, 65, 91, 74, 89, 77])
print("Maths  :", maths)
print("Science:", science)

print("\n--- Mathematical Operations ---")
print("Addition      :", maths + science)
print("Subtraction   :", maths - science)
print("Multiplication:", maths * science)
print("Division      :", np.round(maths / science, 2))

print("\n--- Statistical Functions (Maths marks) ---")
print("Mean              :", np.mean(maths))
print("Median            :", np.median(maths))
print("Minimum           :", np.min(maths))
print("Maximum           :", np.max(maths))
print("Standard Deviation:", round(float(np.std(maths)), 2))
print("Sum               :", np.sum(maths))

print("\n--- Statistical Functions (Science marks) ---")
print("Mean              :", np.mean(science))
print("Median            :", np.median(science))
print("Minimum           :", np.min(science))
print("Maximum           :", np.max(science))
print("Standard Deviation:", round(float(np.std(science)), 2))
print("Sum               :", np.sum(science))
```

### Output

```
Maths  : [78 85 62 90 55 73 88 67 94 81]
Science: [72 80 70 85 60 65 91 74 89 77]

--- Mathematical Operations ---
Addition      : [150 165 132 175 115 138 179 141 183 158]
Subtraction   : [ 6  5 -8  5 -5  8 -3 -7  5  4]
Multiplication: [5616 6800 4340 7650 3300 4745 8008 4958 8366 6237]
Division      : [1.08 1.06 0.89 1.06 0.92 1.12 0.97 0.91 1.06 1.05]

--- Statistical Functions (Maths marks) ---
Mean              : 77.3
Median            : 79.5
Minimum           : 55
Maximum           : 94
Standard Deviation: 12.18
Sum               : 773

--- Statistical Functions (Science marks) ---
Mean              : 76.3
Median            : 75.5
Minimum           : 60
Maximum           : 91
Standard Deviation: 9.61
Sum               : 763
```

# Task 4: Pandas Series & DataFrame

### Explanation

I made a Pandas Series of daily sales with the day names as the index. Then I made a DataFrame of 6 students with ID, name, department and marks, and printed its column names and index. Finally I added a new Grade column (A for 85 and above, B for 70 and above, otherwise C) and printed the updated DataFrame.

### Source code (task4_pandas_series_dataframe.py)

```
# Task 4: Pandas Series & DataFrame
import pandas as pd

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", None)

# Series
series = pd.Series([250, 300, 150, 400, 350],
                   index=["Mon", "Tue", "Wed", "Thu", "Fri"],
                   name="Daily_Sales")
print("Pandas Series:\n", series)

# DataFrame with student information
data = {
    "Student_ID": [101, 102, 103, 104, 105, 106],
    "Name": ["Aarav", "Ananya", "Rohan", "Priya", "Karan", "Sneha"],
    "Department": ["CSE", "ECE", "CSE", "ISE", "ME", "CSE"],
    "Marks": [88, 92, 75, 81, 67, 95],
}
df = pd.DataFrame(data)
print("\nDataFrame:\n", df)
print("\nColumn names:", list(df.columns))
print("Index:", df.index)

# Add a new column
df["Grade"] = ["A" if m >= 85 else "B" if m >= 70 else "C" for m in df["Marks"]]
print("\nUpdated DataFrame:\n", df)
```

### Output

```
Pandas Series:
 Mon    250
Tue    300
Wed    150
Thu    400
Fri    350
Name: Daily_Sales, dtype: int64

DataFrame:
    Student_ID    Name Department  Marks
0         101   Aarav        CSE     88
1         102  Ananya        ECE     92
2         103   Rohan        CSE     75
3         104   Priya        ISE     81
4         105   Karan         ME     67
5         106   Sneha        CSE     95

Column names: ['Student_ID', 'Name', 'Department', 'Marks']
Index: RangeIndex(start=0, stop=6, step=1)

Updated DataFrame:
    Student_ID    Name Department  Marks Grade
0         101   Aarav        CSE     88     A
1         102  Ananya        ECE     92     A
2         103   Rohan        CSE     75     B
3         104   Priya        ISE     81     B
4         105   Karan         ME     67     C
5         106   Sneha        CSE     95     A
```

# Task 5: Reading & Inspecting Data

### Explanation

I read student_performance.csv, which has 50 students and 9 columns, using read_csv(). I looked at the first and last 5 rows with head() and tail(), then checked the shape, column names and data types. info() and describe() gave me the summary. info() also showed that some columns have fewer than 50 values, which means they have missing data.

### Source code (task5_reading_inspecting.py)

```
# Task 5: Reading & Inspecting Data
import os
import pandas as pd

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", None)

BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "student_performance.csv"))

print("First 5 rows:\n", df.head())
print("\nLast 5 rows:\n", df.tail())
print("\nRows and columns:", df.shape)
print("Number of rows:", df.shape[0], "| Number of columns:", df.shape[1])
print("\nColumn names:", list(df.columns))
print("\nData types:\n", df.dtypes)
print("\nInfo:")
df.info()
print("\nDescribe:\n", df.describe().round(2))
```

### Output

```
First 5 rows:
    Student_ID            Name  Gender Department  Attendance_Pct  Study_Hours  Math  Science  English
0         101  Ananya Poojary  Female        ISE            71.0          4.5  67.0     99.0     97.0
1         102    Pooja Kamath    Male        ECE            95.0          3.3  33.0     84.0     95.0
2         103     Arjun Kulal  Female        ISE            77.0          2.0  50.0     52.0     67.0
3         104    Ananya Hegde    Male        ECE            86.0          1.7  60.0      NaN     36.0
4         105    Karan Shetty  Female         ME            75.0          5.1  97.0     64.0     73.0

Last 5 rows:
     Student_ID         Name  Gender Department  Attendance_Pct  Study_Hours  Math  Science  English
45         146  Arjun Kulal  Female        CSE            58.0          NaN  91.0     76.0     61.0
46         147   Manoj Bhat    Male        ECE            89.0          4.6  33.0     51.0     42.0
47         148   Neha Nayak    Male        CSE             NaN          2.8   NaN     96.0     59.0
48         149  Karan Hegde  Female        ECE            90.0          7.6   NaN     66.0     95.0
49         150   Nikhil Rao    Male        ISE            56.0          2.2  51.0     50.0     62.0

Rows and columns: (50, 9)
Number of rows: 50 | Number of columns: 9

Column names: ['Student_ID', 'Name', 'Gender', 'Department', 'Attendance_Pct', 'Study_Hours', 'Math', 'Science', 'English']

Data types:
 Student_ID          int64
Name                 str
Gender               str
Department           str
Attendance_Pct     float64
Study_Hours        float64
Math               float64
Science            float64
English            float64
dtype: object
```

```
Info:
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Student_ID      50 non-null     int64
 1   Name            50 non-null     str
 2   Gender          50 non-null     str
 3   Department      50 non-null     str
 4   Attendance_Pct  46 non-null     float64
 5   Study_Hours     47 non-null     float64
 6   Math            45 non-null     float64
 7   Science         47 non-null     float64
 8   English         46 non-null     float64
dtypes: float64(5), int64(1), str(3)
memory usage: 3.6 KB

Describe:
        Student_ID  Attendance_Pct  Study_Hours   Math  Science  English
count       50.00           46.00        47.00  45.00    47.00    46.00
mean       125.50           76.72         4.49  62.58    63.38    68.91
std         14.58           12.26         2.03  22.83    21.58    18.16
min        101.00           55.00         1.20  30.00    30.00    35.00
25%        113.25           66.25         2.85  40.00    44.50    53.75
50%        125.50           77.00         4.50  58.00    64.00    72.00
75%        137.75           87.00         6.25  82.00    82.00    83.75
max        150.00           98.00         7.70  99.00    99.00    97.00
```

# Task 6: Selecting, Filtering & Sorting Data

### Explanation

On the student data I selected a few columns, and then some rows using iloc and loc. I filtered the students with Math above 85. For multiple conditions I used & and | (CSE students with Math 60 or more and attendance 75 or more, and students scoring above 90 in Science or English). At the end I sorted the data by Math in ascending and descending order.

### Source code (task6_select_filter_sort.py)

```
# Task 6: Selecting, Filtering & Sorting Data
import os
import pandas as pd

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", None)

BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "student_performance.csv"))

# Selecting columns
print("Selected columns (Name, Department, Math):\n",
      df[["Name", "Department", "Math"]].head())

# Selecting rows
print("\nFirst 5 rows using iloc[0:5]:\n", df.iloc[0:5][["Student_ID", "Name"]])
print("\nRows 10-12 using loc[10:12]:\n", df.loc[10:12, ["Student_ID", "Name", "Math"]])

# Filtering on a single condition
high_math = df[df["Math"] > 85]
print("\nStudents with Math > 85:\n", high_math[["Name", "Department", "Math"]])

# Multiple conditions
cond = df[(df["Department"] == "CSE") & (df["Math"] >= 60) & (df["Attendance_Pct"] >= 75)]
print("\nCSE students with Math >= 60 AND Attendance >= 75:\n",
      cond[["Name", "Math", "Attendance_Pct"]])

either = df[(df["Science"] > 90) | (df["English"] > 90)]
print("\nStudents with Science > 90 OR English > 90:\n",
      either[["Name", "Science", "English"]])

# Sorting
asc = df.sort_values("Math", ascending=True)
print("\nSorted by Math (ascending), first 5:\n", asc[["Name", "Math"]].head())
desc = df.sort_values("Math", ascending=False)
print("\nSorted by Math (descending), first 5:\n", desc[["Name", "Math"]].head())
```

### Output

```
Selected columns (Name, Department, Math):
              Name Department  Math
0  Ananya Poojary        ISE  67.0
1    Pooja Kamath        ECE  33.0
2     Arjun Kulal        ISE  50.0
3    Ananya Hegde        ECE  60.0
4    Karan Shetty         ME  97.0

First 5 rows using iloc[0:5]:
    Student_ID            Name
0         101  Ananya Poojary
1         102    Pooja Kamath
2         103     Arjun Kulal
3         104    Ananya Hegde
4         105    Karan Shetty

Rows 10-12 using loc[10:12]:
     Student_ID           Name  Math
10         111     Rahul Bhat  89.0
11         112  Priya Acharya   NaN
12         113     Isha Hegde   NaN

Students with Math > 85:
               Name Department  Math
4     Karan Shetty         ME  97.0
5    Rahul Acharya      CIVIL  99.0
7   Suresh Poojary         ME  92.0
10      Rahul Bhat        ISE  89.0
13     Arjun Kulal         ME  92.0
24   Pooja Poojary      CIVIL  92.0
31   Divya Acharya        CSE  95.0
32      Arjun Bhat         ME  96.0
38    Vikram Nayak        ECE  92.0
45     Arjun Kulal        CSE  91.0
```

```
CSE students with Math >= 60 AND Attendance >= 75:
             Name  Math  Attendance_Pct
19   Sneha Hegde  84.0            95.0
21   Suresh Bhat  76.0            75.0
27  Meera Shetty  82.0            81.0

Students with Science > 90 OR English > 90:
              Name  Science  English
0   Ananya Poojary     99.0     97.0
1     Pooja Kamath     84.0     95.0
8        Rahul Rao     95.0     89.0
16      Ananya Pai     97.0     45.0
19     Sneha Hegde     65.0     95.0
20   Priya Poojary     94.0     82.0
37    Pooja Kamath     94.0     69.0
40    Varun Kamath     98.0      NaN
41     Priya Kulal     51.0     91.0
47      Neha Nayak     96.0     59.0
48     Karan Hegde     66.0     95.0

Sorted by Math (ascending), first 5:
             Name  Math
30  Varun Poojary  30.0
18     Neha Kulal  31.0
34  Ananya Kamath  32.0
1    Pooja Kamath  33.0
46     Manoj Bhat  33.0

Sorted by Math (descending), first 5:
              Name  Math
5    Rahul Acharya  99.0
4     Karan Shetty  97.0
32      Arjun Bhat  96.0
31   Divya Acharya  95.0
7   Suresh Poojary  92.0
```

# Task 7: Handling Missing Values

### Explanation

The student file has 19 missing values spread over 17 rows. I found them with isnull() and counted them for each column. Then I tried two ways of handling them: dropna() removes the rows and leaves only 33 students, while fillna() keeps all 50 rows (I used the median for marks and attendance and the mean for study hours). I printed some rows before and after filling. Handling missing data is important because it can distort averages and totals, break functions and lead to wrong conclusions, so it should be fixed before any analysis.

### Source code (task7_missing_values.py)

```
# Task 7: Handling Missing Values
import os
import pandas as pd

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", None)

BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "student_performance.csv"))

print("BEFORE handling - shape:", df.shape)
print("\nMissing values (isnull):\n", df.isnull().head())
print("\nMissing values per column:\n", df.isnull().sum())
print("Total missing values:", df.isnull().sum().sum())
print("\nRows containing at least one missing value:", df.isnull().any(axis=1).sum())

# Option 1: remove rows that contain missing values
df_dropped = df.dropna()
print("\nAFTER dropna() - shape:", df_dropped.shape)

# Option 2: fill missing values
df_filled = df.copy()
for col in ["Attendance_Pct", "Math", "Science", "English"]:
    df_filled[col] = df_filled[col].fillna(df_filled[col].median())   # skewed/integer-like marks -> median
df_filled["Study_Hours"] = df_filled["Study_Hours"].fillna(df_filled["Study_Hours"].mean())
df_filled["Study_Hours"] = df_filled["Study_Hours"].round(1)

print("\nAFTER fillna() - shape:", df_filled.shape)
print("Missing values per column after filling:\n", df_filled.isnull().sum())

print("\nRows that had missing values - before:\n",
      df[df.isnull().any(axis=1)].head(6))
print("\nSame rows - after filling:\n",
      df_filled.loc[df[df.isnull().any(axis=1)].head(6).index])

print("""
Why handling missing data matters in Data Analytics:
- Missing values bias statistics such as mean, sum and correlation.
- Many functions and ML models fail or silently drop rows when NaN is present.
- Unhandled gaps reduce data quality and lead to wrong business decisions.
- Dropping rows loses information, while filling (mean/median) keeps the dataset size intact.
""")
```

### Output

```
BEFORE handling - shape: (50, 9)

Missing values (isnull):
    Student_ID   Name  Gender  Department  Attendance_Pct  Study_Hours   Math  Science  English
0       False  False   False       False           False        False  False    False    False
1       False  False   False       False           False        False  False    False    False
2       False  False   False       False           False        False  False    False    False
3       False  False   False       False           False        False  False     True    False
4       False  False   False       False           False        False  False    False    False

Missing values per column:
 Student_ID        0
Name               0
Gender             0
Department         0
Attendance_Pct     4
Study_Hours        3
Math               5
Science            3
English            4
dtype: int64
Total missing values: 19

Rows containing at least one missing value: 17

AFTER dropna() - shape: (33, 9)

AFTER fillna() - shape: (50, 9)
Missing values per column after filling:
 Student_ID        0
Name               0
Gender             0
Department         0
Attendance_Pct     0
Study_Hours        0
Math               0
Science            0
English            0
dtype: int64
```

```
Rows that had missing values - before:
     Student_ID           Name  Gender Department  Attendance_Pct  Study_Hours  Math  Science  English
3          104    Ananya Hegde    Male        ECE            86.0          1.7  60.0      NaN     36.0
5          106   Rahul Acharya  Female      CIVIL             NaN          2.2  99.0     65.0     71.0
6          107  Suresh Poojary  Female        CSE            89.0          7.5  40.0     60.0      NaN
9          110    Manoj Kamath  Female      CIVIL            90.0          5.1  82.0     30.0      NaN
11         112   Priya Acharya  Female      CIVIL            87.0          7.7   NaN     46.0     44.0
12         113      Isha Hegde  Female         ME            90.0          4.4   NaN     49.0     83.0

Same rows - after filling:
     Student_ID           Name  Gender Department  Attendance_Pct  Study_Hours  Math  Science  English
3          104    Ananya Hegde    Male        ECE            86.0          1.7  60.0     64.0     36.0
5          106   Rahul Acharya  Female      CIVIL            77.0          2.2  99.0     65.0     71.0
6          107  Suresh Poojary  Female        CSE            89.0          7.5  40.0     60.0     72.0
9          110    Manoj Kamath  Female      CIVIL            90.0          5.1  82.0     30.0     72.0
11         112   Priya Acharya  Female      CIVIL            87.0          7.7  58.0     46.0     44.0
12         113      Isha Hegde  Female         ME            90.0          4.4  58.0     49.0     83.0

Why handling missing data matters in Data Analytics:
- Missing values bias statistics such as mean, sum and correlation.
- Many functions and ML models fail or silently drop rows when NaN is present.
- Unhandled gaps reduce data quality and lead to wrong business decisions.
- Dropping rows loses information, while filling (mean/median) keeps the dataset size intact.
```

# Task 8: Merge, Concatenate, GroupBy & Pivot Table

### Explanation

I merged the student data with departments.csv on the Department column so that every student gets the HOD and block. I joined the two student batches with concat() to get 65 rows. On the combined data I used groupby to find the average marks per department, the count, sum, mean, min and max of Math, and the average by gender. Then I made a pivot table of average marks by department and gender, and another one with the mean and max of each subject.

### Source code (task8_merge_groupby_pivot.py)

```
# Task 8: Merge, Concatenate, GroupBy & Pivot Table
import os
import pandas as pd

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", None)

BASE = os.path.dirname(os.path.abspath(__file__))
students = pd.read_csv(os.path.join(BASE, "student_performance.csv"))
batch2 = pd.read_csv(os.path.join(BASE, "student_performance_batch2.csv"))
depts = pd.read_csv(os.path.join(BASE, "departments.csv"))

print("Departments table:\n", depts)

# ---- Merge on a common column ----
merged = pd.merge(students, depts, on="Department", how="left")
print("\nMerged (students + departments) on 'Department':")
print(merged[["Student_ID", "Name", "Department", "HOD", "Block"]].head())
print("Merged shape:", merged.shape)

# ---- Concatenate ----
combined = pd.concat([students, batch2], ignore_index=True)
print("\nConcatenate batch 1 + batch 2:")
print("Batch 1:", students.shape, "| Batch 2:", batch2.shape, "| Combined:", combined.shape)
print(combined.tail())

# Prepare a total/average column for analysis
combined["Total"] = combined[["Math", "Science", "English"]].sum(axis=1)
combined["Average"] = combined[["Math", "Science", "English"]].mean(axis=1).round(2)

# ---- GroupBy ----
print("\nGroupBy Department - mean of subjects:")
print(combined.groupby("Department")[["Math", "Science", "English"]].mean().round(2))

print("\nGroupBy Department - multiple aggregates on Math:")
print(combined.groupby("Department")["Math"].agg(["count", "sum", "mean", "min", "max"]).round(2))

print("\nGroupBy Gender - average of 'Average':")
print(combined.groupby("Gender")["Average"].mean().round(2))

# ---- Pivot Table ----
pivot = pd.pivot_table(combined, index="Department", columns="Gender",
                       values="Average", aggfunc="mean").round(2)
print("\nPivot table: Department (rows) x Gender (columns), mean Average:")
print(pivot)

pivot2 = pd.pivot_table(combined, index="Department", values=["Math", "Science", "English"],
                        aggfunc=["mean", "max"]).round(2)
print("\nPivot table: Department summary (mean & max per subject):")
print(pivot2)
```

### Output

```
Departments table:
   Department           HOD Block
0         CSE  Dr. R. Menon     A
1         ECE   Dr. S. Iyer     B
2          ME  Dr. K. Reddy     C
3       CIVIL  Dr. A. Patel     D
4         ISE   Dr. M. Nair     A

Merged (students + departments) on 'Department':
   Student_ID            Name Department           HOD Block
0         101  Ananya Poojary        ISE   Dr. M. Nair     A
1         102    Pooja Kamath        ECE   Dr. S. Iyer     B
2         103     Arjun Kulal        ISE   Dr. M. Nair     A
3         104    Ananya Hegde        ECE   Dr. S. Iyer     B
4         105    Karan Shetty         ME  Dr. K. Reddy     C
Merged shape: (50, 11)

Concatenate batch 1 + batch 2:
Batch 1: (50, 9) | Batch 2: (15, 9) | Combined: (65, 9)
    Student_ID            Name  Gender Department  Attendance_Pct  Study_Hours  Math  Science  English
60         161  Vikram Acharya    Male        ISE            61.0          NaN   NaN     44.0     69.0
61         162   Rahul Poojary    Male        ISE             NaN          NaN  89.0     71.0     95.0
62         163    Neha Poojary    Male        ISE            79.0          3.7   NaN     78.0     82.0
63         164        Neha Pai  Female        ISE             NaN          5.8   NaN     65.0     43.0
64         165    Aarav Kamath  Female        ECE            78.0          3.1  44.0     90.0     97.0

GroupBy Department - mean of subjects:
             Math  Science  English
Department
CIVIL       68.64    64.20    74.33
CSE         69.58    70.69    68.46
ECE         51.78    68.80    68.82
ISE         67.42    62.71    71.25
ME          62.55    56.33    65.58
```

```
GroupBy Department - multiple aggregates on Math:
            count    sum   mean   min   max
Department
CIVIL          11  755.0  68.64  42.0  99.0
CSE            12  835.0  69.58  34.0  95.0
ECE             9  466.0  51.78  31.0  92.0
ISE            12  809.0  67.42  30.0  94.0
ME             11  688.0  62.55  32.0  97.0

GroupBy Gender - average of 'Average':
Gender
Female    64.14
Male      67.54
Name: Average, dtype: float64

Pivot table: Department (rows) x Gender (columns), mean Average:
Gender      Female   Male
Department
CIVIL        67.14  69.87
CSE          65.95  69.83
ECE          78.75  61.06
ISE          60.23  73.71
ME           60.17  64.41

Pivot table: Department summary (mean & max per subject):
              mean                    max
           English   Math Science English  Math Science
Department
CIVIL        74.33  68.64   64.20    91.0  99.0    98.0
CSE          68.46  69.58   70.69    96.0  95.0    96.0
ECE          68.82  51.78   68.80    97.0  92.0    97.0
ISE          71.25  67.42   62.71    97.0  94.0    99.0
ME           65.58  62.55   56.33    89.0  97.0    95.0
```

# Task 9: Exporting Data

### Explanation

After filling the missing values I added Total, Average and Result columns (Pass if the average is 40 or more) and saved the data to processed_students.csv using to_csv(). To check it, I read the file back and confirmed that it has 50 rows and 12 columns, no missing values, and the same data as my DataFrame.

### Source code (task9_export_data.py)

```
# Task 9: Exporting Data
import os
import pandas as pd

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", None)

BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "student_performance.csv"))

# Process: fill missing values and add derived columns
for col in ["Attendance_Pct", "Math", "Science", "English"]:
    df[col] = df[col].fillna(df[col].median())
df["Study_Hours"] = df["Study_Hours"].fillna(df["Study_Hours"].mean()).round(1)
df["Total"] = df[["Math", "Science", "English"]].sum(axis=1)
df["Average"] = (df["Total"] / 3).round(2)
df["Result"] = df["Average"].apply(lambda x: "Pass" if x >= 40 else "Fail")

# Export
out_path = os.path.join(BASE, "processed_students.csv")
df.to_csv(out_path, index=False)
print("Exported to:", out_path)

# Verify by reading the exported file back
check = pd.read_csv(out_path)
print("\nVerification - shape of exported file:", check.shape)
print("Columns:", list(check.columns))
print("Missing values in exported file:", int(check.isnull().sum().sum()))
print("Identical to processed DataFrame:", check.round(2).equals(df.round(2)))
print("\nFirst 5 rows of exported file:\n", check.head())
```

### Output

```
Exported to: /home/claude/Week2_Assignment/processed_students.csv

Verification - shape of exported file: (50, 12)
Columns: ['Student_ID', 'Name', 'Gender', 'Department', 'Attendance_Pct', 'Study_Hours', 'Math', 'Science', 'English', 'Total', 'Average', 'Result']
Missing values in exported file: 0
Identical to processed DataFrame: True

First 5 rows of exported file:
    Student_ID           Name  Gender Department  Attendance_Pct  Study_Hours  Math  Science  English  Total  Average  \
0         101  Ananya Poojary  Female        ISE            71.0          4.5  67.0     99.0     97.0  263.0    87.67
1         102    Pooja Kamath    Male        ECE            95.0          3.3  33.0     84.0     95.0  212.0    70.67
2         103     Arjun Kulal  Female        ISE            77.0          2.0  50.0     52.0     67.0  169.0    56.33
3         104    Ananya Hegde    Male        ECE            86.0          1.7  60.0     64.0     36.0  160.0    53.33
4         105    Karan Shetty  Female         ME            75.0          5.1  97.0     64.0     73.0  234.0    78.00

  Result
0   Pass
1   Pass
2   Pass
3   Pass
4   Pass
```

# Task 10: Mini Project: Sales Data Analysis

### Explanation

For the mini project I used a sales dataset of 120 orders. I loaded and inspected it, filled the missing Quantity, Discount and Region values, and calculated Revenue using NumPy. Then I filtered and sorted the data and grouped it by category, region and customer type. I also made a Category x Region pivot table and saved the cleaned data as sales_cleaned.csv.

### Source code (task10_mini_project_sales_analysis.py)

```
# Task 10: Mini Data Analysis Project - Sales Dataset
import os
import numpy as np
import pandas as pd

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", None)

BASE = os.path.dirname(os.path.abspath(__file__))

# 1. Load
df = pd.read_csv(os.path.join(BASE, "sales_data.csv"), parse_dates=["Date"])
print("=== 1. LOAD ===")
print(df.head())

# 2. Inspect
print("\n=== 2. INSPECT ===")
print("Shape:", df.shape)
df.info()
print(df.describe().round(2))

# 3. Missing values
print("\n=== 3. MISSING VALUES ===")
print(df.isnull().sum())
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["Discount_Pct"] = df["Discount_Pct"].fillna(0)            # no discount recorded -> 0
df["Region"] = df["Region"].fillna(df["Region"].mode()[0])   # categorical -> most frequent
print("After cleaning:\n", df.isnull().sum())

# Derived columns with NumPy
df["Gross"] = df["Unit_Price"] * df["Quantity"]
df["Revenue"] = np.round(df["Gross"] * (1 - df["Discount_Pct"] / 100), 2)
df["Month"] = df["Date"].dt.month_name()

# 4. Selecting & filtering
print("\n=== 4. SELECT & FILTER ===")
print(df[["Order_ID", "Category", "Region", "Revenue"]].head())
big = df[(df["Revenue"] > 50000) & (df["Customer_Type"] == "Returning")]
print("\nReturning-customer orders with revenue > 50,000:", len(big))
print(big[["Order_ID", "Category", "Region", "Revenue"]].head())

# 5. Sorting
print("\n=== 5. SORTING ===")
print("Top 5 orders by revenue:\n",
      df.sort_values("Revenue", ascending=False)[["Order_ID", "Category", "Region", "Revenue"]].head())

# 6. GroupBy
print("\n=== 6. GROUPBY ===")
cat = df.groupby("Category")["Revenue"].agg(["count", "sum", "mean"]).round(2).sort_values("sum", ascending=False)
print("Revenue by category:\n", cat)
reg = df.groupby("Region")["Revenue"].sum().round(2).sort_values(ascending=False)
print("\nRevenue by region:\n", reg)
cust = df.groupby("Customer_Type")["Revenue"].agg(["count", "sum", "mean"]).round(2)
print("\nRevenue by customer type:\n", cust)

# 7. Pivot table
print("\n=== 7. PIVOT TABLE ===")
pivot = pd.pivot_table(df, index="Category", columns="Region", values="Revenue",
                       aggfunc="sum", fill_value=0, margins=True, margins_name="Total").round(0)
print(pivot)

# 8. Insights
print("\n=== 8. INSIGHTS ===")
top_cat = cat.index[0]
print(f"- Total revenue: {df['Revenue'].sum():,.0f} across {len(df)} orders "
```

```
      f"(average order value {df['Revenue'].mean():,.0f}).")
print(f"- Top category: {top_cat} contributes {cat['sum'].iloc[0] / df['Revenue'].sum() * 100:.1f}% of revenue.")
print(f"- Top region: {reg.index[0]} ({reg.iloc[0]:,.0f}); weakest region: {reg.index[-1]} ({reg.iloc[-1]:,.0f}).")
print(f"- Returning customers generate {cust.loc['Returning', 'sum'] / df['Revenue'].sum() * 100:.1f}% of revenue.")
monthly = df.groupby("Month")["Revenue"].sum()
print(f"- Best month: {monthly.idxmax()} ({monthly.max():,.0f}); weakest month: {monthly.idxmin()} ({monthly.min():,.0 \
    f}).")
disc = df.groupby("Discount_Pct")["Quantity"].mean().round(2)
print("- Average quantity by discount level:", disc.to_dict())

# 9. Export
df.to_csv(os.path.join(BASE, "sales_cleaned.csv"), index=False)
print("\n=== 9. EXPORT ===")
print("Saved sales_cleaned.csv with shape", pd.read_csv(os.path.join(BASE, "sales_cleaned.csv")).shape)
```

## Output

```
=== 1. LOAD ===
   Order_ID       Date Region     Category  Unit_Price  Quantity  Discount_Pct Customer_Type
0      5001 2026-08-31  South         Home      6692.0       4.0          10.0     Returning
1      5002 2026-03-01   East     Clothing      1080.0       NaN          10.0     Returning
2      5003 2026-01-05   West  Electronics     23456.0       5.0           5.0     Returning
3      5004 2026-10-01  South         Home      7690.0       4.0           5.0     Returning
4      5005 2026-03-28  North  Electronics     42053.0       5.0           0.0           New

=== 2. INSPECT ===
Shape: (120, 8)
<class 'pandas.DataFrame'>
RangeIndex: 120 entries, 0 to 119
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Order_ID       120 non-null    int64
 1   Date           120 non-null    datetime64[us]
 2   Region         116 non-null    str
 3   Category       120 non-null    str
 4   Unit_Price     120 non-null    float64
 5   Quantity       114 non-null    float64
 6   Discount_Pct   115 non-null    float64
 7   Customer_Type  120 non-null    str
dtypes: datetime64[us](1), float64(3), int64(1), str(3)
memory usage: 7.6 KB
       Order_ID                Date  Unit_Price  Quantity  Discount_Pct
count    120.00                 120      120.00    114.00        115.00
mean    5060.50  2026-05-24 15:12:00     8915.33      3.28          7.61
min     5001.00  2026-01-02 00:00:00      111.00      1.00          0.00
25%     5030.75  2026-02-26 12:00:00     1164.50      2.00          5.00
50%     5060.50  2026-05-27 12:00:00     3087.50      4.00          5.00
75%     5090.25  2026-08-12 06:00:00     8626.50      4.75         10.00
max     5120.00  2026-10-22 00:00:00    44641.00      5.00         15.00
std       34.79                  NaN    11914.69      1.42          5.31
```

```
=== 3. MISSING VALUES ===
Order_ID         0
Date             0
Region           4
Category         0
Unit_Price       0
Quantity         6
Discount_Pct     5
Customer_Type    0
dtype: int64
After cleaning:
 Order_ID         0
Date             0
Region           0
Category         0
Unit_Price       0
Quantity         0
Discount_Pct     0
Customer_Type    0
dtype: int64

=== 4. SELECT & FILTER ===
   Order_ID     Category Region   Revenue
0      5001         Home  South   24091.2
1      5002     Clothing   East    3888.0
2      5003  Electronics   West  111416.0
3      5004         Home  South   29222.0
4      5005  Electronics  North  210265.0

Returning-customer orders with revenue > 50,000: 11
    Order_ID     Category Region   Revenue
2       5003  Electronics   West  111416.0
11      5012  Electronics  South  178564.0
35      5036  Electronics   East   63137.0
36      5037  Electronics  North   71839.0
42      5043  Electronics   East   69518.0
```

```
=== 5. SORTING ===
Top 5 orders by revenue:
      Order_ID     Category Region    Revenue
4        5005  Electronics  North  210265.00
9        5010  Electronics   East  195225.00
11       5012  Electronics  South  178564.00
67       5068  Electronics   West  166791.25
117      5118  Electronics   East  153054.00

=== 6. GROUPBY ===
Revenue by category:
              count         sum      mean
Category
Electronics      31  2492944.30  80417.56
Home             31   471546.10  15211.16
Clothing         26   176193.95   6776.69
Grocery          32    88490.95   2765.34

Revenue by region:
 Region
East     1133588.55
North     746587.90
West      712327.90
South     636670.95
Name: Revenue, dtype: float64

Revenue by customer type:
                 count         sum      mean
Customer_Type
New                 57  1505748.85  26416.65
Returning           63  1723426.45  27355.98
```

```
=== 7. PIVOT TABLE ===
Region            East     North     South      West      Total
Category
Clothing       61997.0   44997.0   38700.0   30500.0   176194.0
Electronics   841268.0  627184.0  421337.0  603154.0  2492944.0
Grocery        17967.0   14092.0   35387.0   21045.0    88491.0
Home          212356.0   60315.0  141247.0   57628.0   471546.0
Total        1133589.0  746588.0  636671.0  712328.0  3229175.0

=== 8. INSIGHTS ===
- Total revenue: 3,229,175 across 120 orders (average order value 26,910).
- Top category: Electronics contributes 77.2% of revenue.
- Top region: East (1,133,589); weakest region: South (636,671).
- Returning customers generate 53.4% of revenue.
- Best month: January (914,960); weakest month: August (114,967).
- Average quantity by discount level: {0.0: 3.07, 5.0: 3.44, 10.0: 3.48, 15.0: 3.22}

=== 9. EXPORT ===
Saved sales_cleaned.csv with shape (120, 11)
```

### Key Findings

- The 120 orders brought in a total revenue of 3,229,175, which is an average of about 26,910 per order.
- Electronics is the top category and gives 77.2% of the total revenue, because its unit prices are much higher than the other categories.

- East is the best region (1,133,589) and South is the weakest (636,671).
- Returning customers placed more orders and gave 53.4% of the revenue.
- January was the best month (914,960) and August was the weakest (114,967).
- The average quantity per order stays between about 3.1 and 3.5 at every discount level, so discounts do not seem to increase the quantity much.

# Datasets Used

These are sample datasets that I prepared for this assignment. I added some missing values on purpose so that I could practise the cleaning tasks. All the files are also in my Google Drive folder.

### student_performance.csv

Tasks 5 to 9: 50 students with marks, attendance and study hours (shape: 50 rows x 9 columns)

| | Student_ID | Name | Gender | Department | Attendance_Pct | Study_Hours | Math | Science | English |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 101 | Ananya Poojary | Female | ISE | 71.0 | 4.5 | 67.0 | 99.0 | 97.0 |
| 1 | 102 | Pooja Kamath | Male | ECE | 95.0 | 3.3 | 33.0 | 84.0 | 95.0 |
| 2 | 103 | Arjun Kulal | Female | ISE | 77.0 | 2.0 | 50.0 | 52.0 | 67.0 |
| 3 | 104 | Ananya Hegde | Male | ECE | 86.0 | 1.7 | 60.0 | NaN | 36.0 |
| 4 | 105 | Karan Shetty | Female | ME | 75.0 | 5.1 | 97.0 | 64.0 | 73.0 |

### student_performance_batch2.csv

Task 8: 15 more students with the same columns (used for concat) (shape: 15 rows x 9 columns)

| | Student_ID | Name | Gender | Department | Attendance_Pct | Study_Hours | Math | Science | English |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 151 | Sneha Pai | Male | ISE | 99.0 | 7.0 | 85.0 | 86.0 | NaN |
| 1 | 152 | Priya Kamath | Female | CSE | 69.0 | 4.2 | NaN | NaN | NaN |
| 2 | 153 | Rohan Pai | Male | CSE | NaN | 3.7 | 76.0 | 67.0 | 96.0 |
| 3 | 154 | Aarav Poojary | Male | ISE | 81.0 | 5.5 | 94.0 | 83.0 | 74.0 |
| 4 | 155 | Ananya Rao | Male | CSE | 90.0 | 2.9 | 89.0 | 74.0 | 76.0 |

### departments.csv

Task 8: department, HOD and block (used for merge) (shape: 5 rows x 3 columns)

| | Department | HOD | Block |
|---|---|---|---|
| 0 | CSE | Dr. R. Menon | A |
| 1 | ECE | Dr. S. Iyer | B |
| 2 | ME | Dr. K. Reddy | C |
| 3 | CIVIL | Dr. A. Patel | D |
| 4 | ISE | Dr. M. Nair | A |

### sales_data.csv

Task 10: 120 sales orders with region, category, price, quantity and discount (shape: 120 rows x 8 columns)

| | Order_ID | Date | Region | Category | Unit_Price | Quantity | Discount_Pct | Customer_Type |
|---|---|---|---|---|---|---|---|---|
| 0 | 5001 | 2026-08-31 | South | Home | 6692.0 | 4.0 | 10.0 | Returning |
| 1 | 5002 | 2026-03-01 | East | Clothing | 1080.0 | NaN | 10.0 | Returning |
| 2 | 5003 | 2026-01-05 | West | Electronics | 23456.0 | 5.0 | 5.0 | Returning |
| 3 | 5004 | 2026-10-01 | South | Home | 7690.0 | 4.0 | 5.0 | Returning |
| 4 | 5005 | 2026-03-28 | North | Electronics | 42053.0 | 5.0 | 0.0 | New |